Chandigarh Engineering College Jhanjeri
Mohali-140307
Department of Artificial Intelligence (AI) and Data Sciences

ASSIGNMENT - 1

Semester: 4th **Total Marks:** 10
Date of Assignment: 04-02-2026 **Submission Deadline:** 09-02-2026

Q.NO.	Statement	Bloom's Taxonomy Level (PAQIC)	CO/CO's Attained	Marks
1.	Demonstrate the role of the CPU, memory, and I/O subsystems in a computer system.	LEVEL II	CO1	2.5
2.	Convert −25.125 to binary and represent it in IEEE 754 32-bit and 64-bit floating-point formats, showing the sign, exponent, and mantissa.	LEVEL II	CO1	2.5
3.	Demonstrate Booth's Algorithm with an example.	LEVEL II	CO1	2.5
4.	Describe working of DMA with the help of suitable example.	LEVEL II	CO2	2.5

PAQIC (Program Assessment & Quality Improvement Cell) REMARKS

HOD's Signature **PAQIC Member's Signature** **Subject Teacher's Signature**

CAO Assignment 1 Solution

1. Role of the CPU (Central Processing Unit)

Functions

The CPU is the **brain of the computer system**. It performs:

- **Instruction fetch, decode, and execute**
- **Arithmetic and logical operations**
- **Control and coordination** of all system components

Internal Components

- **ALU** – performs calculations
- **Control Unit** – generates control signals
- **Registers** – fast temporary storage

Demonstration:

The CPU fetches an instruction from memory, decodes it, executes it, and directs data transfers with memory and I/O devices.

2. Role of Memory Subsystem

Functions

The memory subsystem:

- Stores **program instructions**
- Stores **input data and intermediate results**
- Stores **final output before transfer**

Types

- **Primary memory (RAM)** – fast, volatile
- **Cache memory** – very fast, small
- **Secondary memory** – non-volatile (HDD/SSD)

Demonstration:

Memory supplies instructions and data to the CPU and receives processed results.

3. Role of I/O Subsystem

Functions

The I/O subsystem enables:

- Communication between the computer and **external devices**
- Input of data and output of results
- Data transfer using **programmed I/O, interrupts, or DMA**

Components

- I/O controllers
- Peripheral devices (keyboard, disk, display, network)

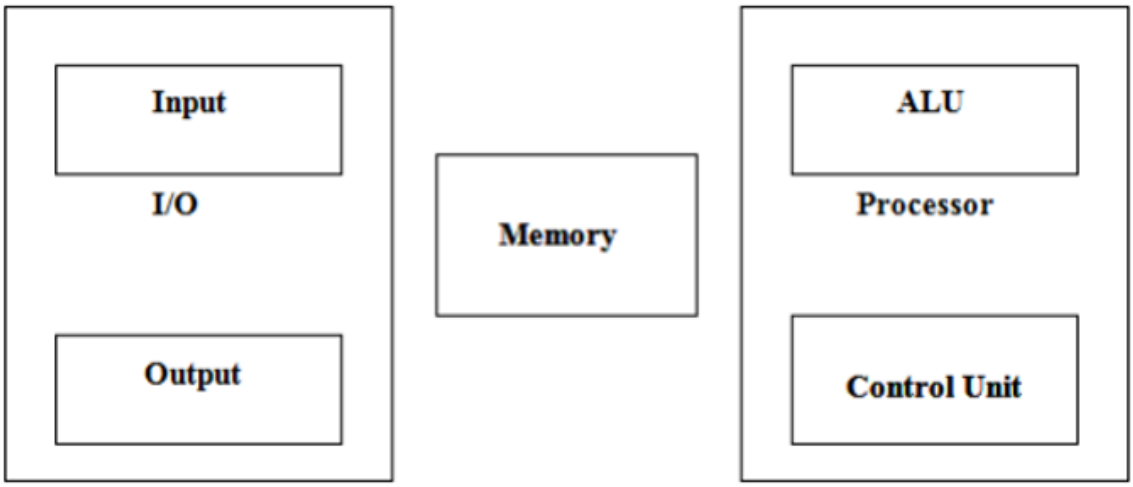
Demonstration:

I/O subsystem receives commands from the CPU and transfers data between peripherals and memory.

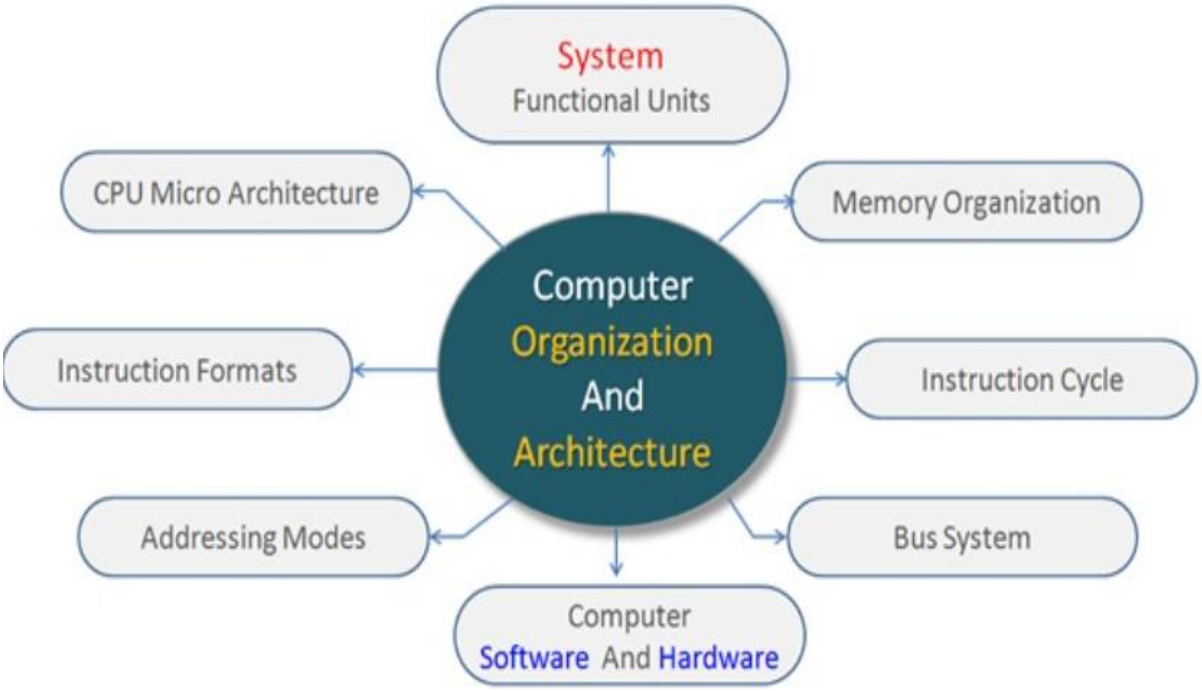
4. Combined Working (Demonstration with Example)

Example: Reading data from keyboard and displaying result

1. **I/O subsystem** captures input from the keyboard.
2. Data is transferred to **memory**.
3. **CPU** fetches data from memory and processes it.
4. Result is stored back into **memory**.
5. **I/O subsystem** sends the output to the display.



Functional units of computer



2. Compare the IEEE 754 32-bit and 64-bit floating-point representations of −45.125 in terms of sign, exponent, and mantissa.

Step 1: Convert −45.125 to Binary

(a) Ignore the sign first

Work with +45.125.

(b) Convert integer part (45) to binary

$45_{10} = 101101_2$

(c) Convert fractional part (0.125) to binary

$0.125 \times 2 = 0.25 \rightarrow 0$

$0.25 \times 2 = 0.5 \rightarrow 0$

$0.5 \times 2 = 1.0 \rightarrow 1$

So,

$0.125_{10} = 0.001_2$

(d) Combine

$45.125_{10} = 101101.001_2$

Step 2: Normalize the Binary Number

IEEE 754 uses normalized form:

$1.xxxxx \times 2^n$

Shift the binary point:

$101101.001_2 = 1.01101001_2 \times 2^5$

So:
Normalized mantissa = 1.01101001
Exponent (actual) = 5
Step 3: Sign Bit
Since the number is negative (−45.125):
Sign bit = 1
IEEE 754 Single Precision (32-bit)
Format

Field	Bits
Sign	1
Exponent	8
Mantissa	23

(a) Exponent
Bias = 127
Stored exponent = 5 + 127 = 132
$132_{10} = 10000100_2$
(b) Mantissa
Drop the leading 1 from normalized form:
Normalized: 1.01101001
Mantissa: 01101001000000000000000 (23 bits)
(c) Final 32-bit Representation
Sign = 1
Exponent = 10000100
Mantissa = 01101001000000000000000
IEEE 754 (32-bit)
1 10000100 01101001000000000000000
IEEE 754 Double Precision (64-bit)
Format

Field	Bits
Sign	1
Exponent	11
Mantissa	52

(a) Exponent
Bias = 1023
Stored exponent = 5 + 1023 = 1028
$1028_{10} = 10000000100_2$
(b) Mantissa
Same fractional bits, padded to 52 bits:
0110100100000000000000000000000000000000000000000000
(c) Final 64-bit Representation
Sign = 1

Exponent = 10000000100
Mantissa = 0110100100000000000000000000000000000000000000000000
IEEE 754 (64-bit)
1 10000000100 0110100100000000000000000000000000000000000000000000

3. Ans. Booth's Algorithm is used for multiplication of two signed binary numbers using 2's complement representation. It efficiently handles both positive and negative multipliers.

Steps of Booth's Algorithm

1. Initialize Registers:

o Multiplier (Q) and Multiplicand (M) are loaded.

o Q_{-1} (extra bit for the multiplier) is set to 0.

o Product (A) is initialized to 0.

o Set the number of bits n (equal to the bit-length of the multiplier).

2. Check the Last Two Bits (Q_0 and Q_{-1}):

o If (Q_0, Q_{-1}) = (1,0): A = A - M

o If (Q_0, Q_{-1}) = (0,1): A = A + M

o If (Q_0, Q_{-1}) = (0,0) or (1,1): Do nothing

3. Arithmetic Right Shift (ARS):

o Shift A, Q, and Q_{-1} right by 1 bit.

4. Repeat Steps 2 and 3 for n times.

5. The final content of (A, Q) gives the product.

Ex.

Example: Multiply (-3) × (4) using Booth's Algorithm

Step 1: Represent Numbers in Binary (5-bit representation)

M=−3=111012(2's complement)

Q=4=001002Q = 4 = 00100_2Q=4=001002

Q−1=0, A=000002Q_{-1} = 0, A = 00000_2Q−1=0, A=000002

Step A (5 bits) Q (5 bits) Q_{-1} Operation

Initial 00000 00100 0 Load values

1 00000 00100 0 (Q_0, Q_{-1}) = (0,0) → No Operation

Shift 00000 00010 0 Arithmetic Right Shift

2 00000 00010 0 (Q_0, Q_{-1}) = (0,0) → No Operation

Shift 00000 00001 0 Arithmetic Right Shift

3 00000 00001 0 (Q_0, Q_{-1}) = (0,0) → No Operation

Step A (5 bits) Q (5 bits) Q_{-1} Operation

Shift 00000 00000 1 Arithmetic Right Shift

4 00000 00000 1 (Q_0, Q_{-1}) = (0,1) → A = A + M (00000 + 11101 = 11101)

Shift 11110 00000 0 Arithmetic Right Shift

Final Result 111100 (-12 in decimal)
Final Answer:
(−3)×(4)=−1210(-3) × (4) = -12_{10} (−3)×(4)=−1210 Binary Result: 1111002\ text {Binary Result: }
111100_2Binary Result: 1111002
(-12 in 2's complement form)

4. **Direct Memory Access Controller (DMAC)**

Definition

A **Direct Memory Access Controller (DMAC)** is a hardware unit that allows **I/O devices to transfer data directly to or from main memory without continuous CPU involvement**. This significantly improves system performance, especially for **large data transfers**.

Why DMAC is Needed

Without DMA:

- CPU must handle **every byte/word transfer**
- CPU remains busy doing data movement
- Overall system performance drops

With DMA:

- CPU initiates the transfer once
- DMAC handles the entire data block
- CPU is free to perform other tasks

Basic Components of DMAC

1. **DMA Address Register**
    - Stores the starting memory address for transfer
2. **DMA Count Register**
    - Stores the number of bytes/words to be transferred
3. **DMA Control Register**
    - Specifies type of operation (read/write)
    - Enables DMA mode
4. **DMA Status Register**
    - Indicates completion or error
5. **Control Logic**
    - Manages bus requests and acknowledgments

Working of DMAC (Step-by-Step)

1. **CPU initializes DMAC**
   - Loads memory address, count, and direction of transfer
2. **I/O device requests DMA**
   - Sends DMA request (DRQ) to DMAC
3. **DMAC requests system bus**
   - Sends HOLD signal to CPU
4. **CPU grants bus control**
   - Sends HLDA (Hold Acknowledge)
5. **Data transfer begins**
   - DMAC transfers data directly between memory and I/O device
6. **Transfer completion**
   - DMAC updates status and sends interrupt to CPU

Advantages of DMAC

1. **Reduces CPU workload**
   - CPU is free for computation
2. **Faster data transfer**
   - Ideal for high-speed devices (disk, network)
3. **Improves system throughput**
   - Parallel processing of CPU and I/O
4. **Efficient for large data blocks**
   - Minimal overhead per transfer

Disadvantages of DMAC

1. **Increased hardware complexity**
   - Additional controller logic required
2. **Bus contention**
   - CPU and DMAC compete for system bus
3. **CPU is paused temporarily**
   - Especially in burst mode

4. **Not efficient for small data transfers**
    - Setup overhead may outweigh benefits

Types (Modes) of DMA

1. Burst Mode DMA

- DMAC transfers **entire block at once**
- CPU is halted during transfer

Pros: Fastest
Cons: CPU blocked for longer time

2. Cycle Stealing DMA

- DMAC transfers **one word at a time**
- Steals bus cycles from CPU

Pros: CPU still executes
Cons: CPU speed slightly reduced

3. Transparent (Hidden) DMA

- DMAC uses bus **only when CPU is idle**

Pros: No CPU interference
Cons: Slow transfer rate

4. Demand Mode DMA

- DMAC continues transfer **as long as device requests**

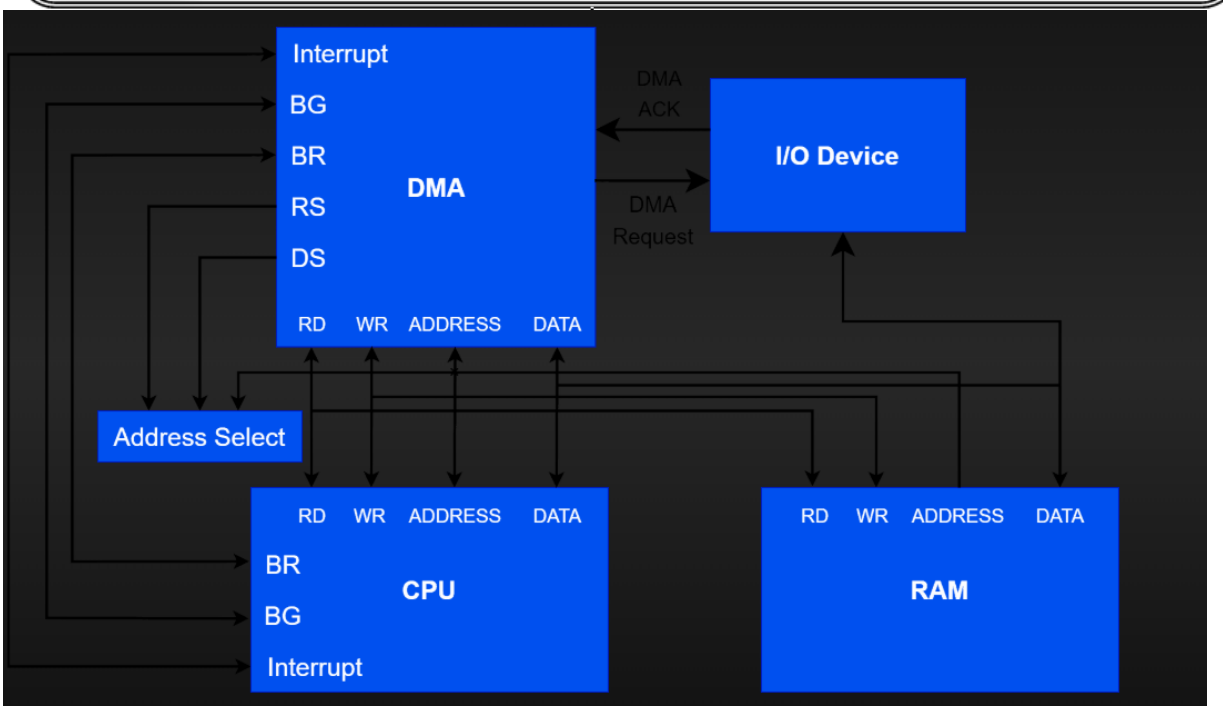
Pros: Flexible control
Cons: Complex logic

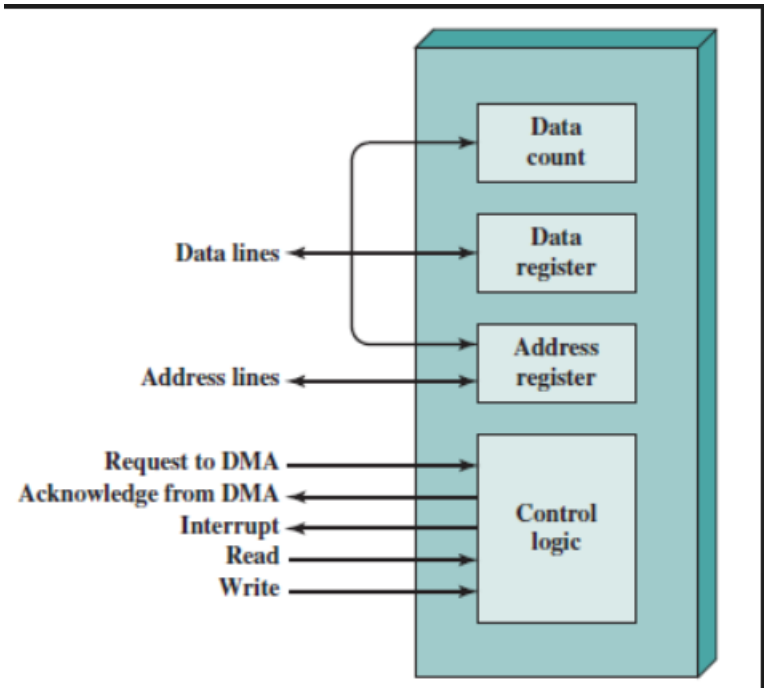
Summary Table

DMA Type	CPU Involvement	Speed
Burst Mode	CPU halted	Very High
Cycle Stealing	Partial CPU access	Medium
Transparent	CPU not affected	Low
Demand Mode	Device-controlled	High

Conclusion

The **DMAC is essential for high-performance systems**, enabling fast and efficient data transfer between memory and peripherals.
Despite added complexity, its advantages far outweigh its limitations—making DMA a core feature in **modern computer architectures**.


Interrupt
BG
BR
RS
DS
DMA
RD WR ADDRESS DATA
DMA ACK
DMA Request
I/O Device
Address Select
RD WR ADDRESS DATA
BR
BG
Interrupt
CPU
RD WR ADDRESS DATA
RAM


Data count
Data lines
Data register
Address lines
Address register
Request to DMA
Acknowledge from DMA
Interrupt
Read
Write
Control logic